

WEEK-6

Task-1: To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

Code:

```
GNU nano 8.7.1                                history.c *
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_CMD 100

char history[MAX_HISTORY][MAX_CMD];
int count = 0;

void add_history(char *cmd) {
    if (count < MAX_HISTORY) {
        strcpy(history[count], cmd);
        count++;
    } else {
        for (int i = 1; i < MAX_HISTORY; i++)
            strcpy(history[i - 1], history[i]);

        strcpy(history[MAX_HISTORY - 1], cmd);
    }
}

void show_history() {
    printf("\nCommand History:\n");

    for (int i = 0; i < count; i++)
        printf("%d  %s\n", i + 1, history[i]);
}

int main() {
    char command[MAX_CMD];

    printf("Enter commands (type 'history' to display, 'exit' to quit):\n");

    while (1) {
        printf("shell$ ");
        fgets(command, sizeof(command), stdin);

        ^G Help      ^O Write Out    ^F Where Is    ^K Cut         ^T Execute
        ^X Exit      ^R Read File    ^\ Replace     ^U Paste       ^J Justify
```

Output:

```
shell exited.  
the_boss@DESKTOP-I8C8AJG:~/skill$ ./history  
Enter commands (type 'history' to display, 'exit' to quit):  
shell$ ls  
shell$ pwd  
shell$ whoami  
shell$ history  
  
Command History:  
1  ls  
2  pwd  
3  whoami  
shell$ |
```

Report: The program stores executed commands in a fixed-size history buffer.

It maintains command order and removes the oldest entry when capacity is exceeded.

The history command retrieves and displays previously stored commands.

The program was tested successfully for storage, retrieval, and capacity handling

Task-2: To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

Code:

```
GNU nano 8.7.1 pipeline_structure.c *
#include <stdio.h>
#include <string.h>

#define MAX_COMMANDS 5
#define MAX_LENGTH 100

int main() {
    char commands[MAX_COMMANDS][MAX_LENGTH];
    int n;

    printf("Enter number of commands: ");
    scanf("%d", &n);
    getchar();

    if (n < 1 || n > MAX_COMMANDS) {
        printf("Invalid number of commands.\n");
        return 1;
    }

    printf("Enter commands in pipeline order:\n");

    for (int i = 0; i < n; i++) {
        printf("Command %d: ", i + 1);
        fgets(commands[i], MAX_LENGTH, stdin);
        commands[i][strcspn(commands[i], "\n")] = '\0';
    }

    printf("\nPipeline Layout:\n");

    for (int i = 0; i < n; i++) {
        printf("%s", commands[i]);

        if (i < n - 1)
            printf(" | ");
    }

    printf("\n");
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify

Output:

```
the_boss@DESKTOP-I8C8AJG:~/skill$ ./pipeline_structure
Enter number of commands: 3
Enter commands in pipeline order:
Command 1: ls
Command 2: grep txt
Command 3: wc -l

Pipeline Layout:
ls | grep txt | wc -l
the_boss@DESKTOP-I8C8AJG:~/skill$ |
```

Report: The program creates a structure for storing commands in pipeline order. It maintains the execution sequence and represents the connection between commands. The pipeline layout is validated before execution. The program was tested successfully with different command configurations.

Task-3: To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

Code:

```
GNU nano 8.7.1                                pipe.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int fd[2];
    pid_t p1, p2;

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    p1 = fork();

    if (p1 == 0) {
        dup2(fd[1], STDOUT_FILENO);

        close(fd[0]);
        close(fd[1]);

        execlp("ls", "ls", NULL);

        perror("ls");
        exit(1);
    }

    p2 = fork();

    if (p2 == 0) {
        dup2(fd[0], STDIN_FILENO);

        close(fd[0]);
        close(fd[1]);

        execlp("wc", "wc", "-l", NULL);
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^\\ Replace	^U Paste	^J Justify

Output:

```
the_boss@DESKTOP-I8C8AJG:~/skill$ gcc pipe.c -o pipe
the_boss@DESKTOP-I8C8AJG:~/skill$ ./pipe
14
Pipeline completed successfully.
the_boss@DESKTOP-I8C8AJG:~/skill$ |
```

Report: The program creates a pipe to establish communication between two processes. The output of the first command is redirected to the input of the second command. Unused file descriptors are closed and both processes are synchronized using `waitpid()`. The program was tested successfully to verify data flow through the pipe

Task-4: To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines

Code:

```
GNU nano 8.7.1 multiple_pipes.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int pipe1[2], pipe2[2];
    pid_t p1, p2, p3;

    pipe(pipe1);
    pipe(pipe2);

    p1 = fork();

    if (p1 == 0) {
        dup2(pipe1[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("ls", "ls", NULL);

        perror("ls");
        exit(1);
    }

    p2 = fork();

    if (p2 == 0) {
        dup2(pipe1[0], STDIN_FILENO);
        dup2(pipe2[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^\\ Replace	^U Paste	^J Justify

Output:

```
the_boss@DESKTOP-I8C8AJG:~/skill$ nano multiple_pipes.c
the_boss@DESKTOP-I8C8AJG:~/skill$ gcc multiple_pipes.c -o multiple_pipes
the_boss@DESKTOP-I8C8AJG:~/skill$ ./multiple_pipes
11
Multiple pipeline execution completed.
the_boss@DESKTOP-I8C8AJG:~/skill$ |
```

Report: The program implements multiple pipes to connect several processes in sequence. File descriptor arrays are managed to redirect input and output between processes. Multiple child processes are launched and synchronized before the program terminates. The program was tested successfully using a long pipeline with multiple commands.